

Bloc 4

Implémenter des méthodes d'intelligence artificielle pour modéliser et prédire de nouveaux comportements et usages

Projet support : **data_science_project** — maintenance prédictive industrielle

Julien TREMONT-RAIMI • Antoine VANDEPLANQUE • Promotion 2024–2026

Cette partie démontre l'acquisition des cinq compétences du Bloc 4 du référentiel RNCP36739 [1] à travers un projet complet de **maintenance prédictive industrielle**. Chaque étape couvre une ou plusieurs sous-compétences du référentiel, synthétisées dans le tableau de validation ci-dessous.

1 Compétences visées

Le référentiel RNCP36739 définit cinq sous-compétences pour le Bloc 4, que ce dossier couvre intégralement à travers le **data science project**. Ce bloc porte sur l'exploitation des données pour produire des modèles prédictifs à valeur métier, du reporting analytique et de la visualisation. Il couvre le cycle complet du Machine Learning : préparation des données, construction de pipelines de transformation, entraînement comparatif de modèles, évaluation rigoureuse et mise en service. Le tableau de preuves d'acquisition présenté en fin de dossier (section 10.1) récapitule, pour chacune, la réalisation concrète qui la démontre.

- Extraire des données en provenance de systèmes d'information variés pour les exploiter et les analyser en utilisant les outils professionnels courants.
- Préparer les données en les transformant et en les nettoyant pour faire l'analyse et le reporting selon les besoins des différents métiers.
- Élaborer une communication infographique visuelle en construisant des tableaux de bord interactifs afin de communiquer les résultats d'analyse et d'assurer l'extraction de connaissances en temps réel.
- Développer un modèle prédictif pour identifier de nouveaux comportements et usages en implémentant des algorithmes d'apprentissage automatique supervisés.
- Évaluer la performance d'un modèle de machine learning en analysant ses résultats et en les comparant avec d'autres modèles afin d'implémenter la solution la plus convenable à un cas d'usage.

2 Contexte et enjeux

Le projet **data science** traite de la **maintenance prédictive industrielle**. Les machines industrielles sont équipées de capteurs (vibrations, température moteur, vitesse de rotation, pression, heures de vie résiduelle) qui produisent un flux continu de mesures. À partir de ces signaux, nous traitons deux tâches de prédiction au moyen de scikit-learn [2] :

- **Classification binaire** — prédire si une panne surviendra dans les prochaines 24 heures.

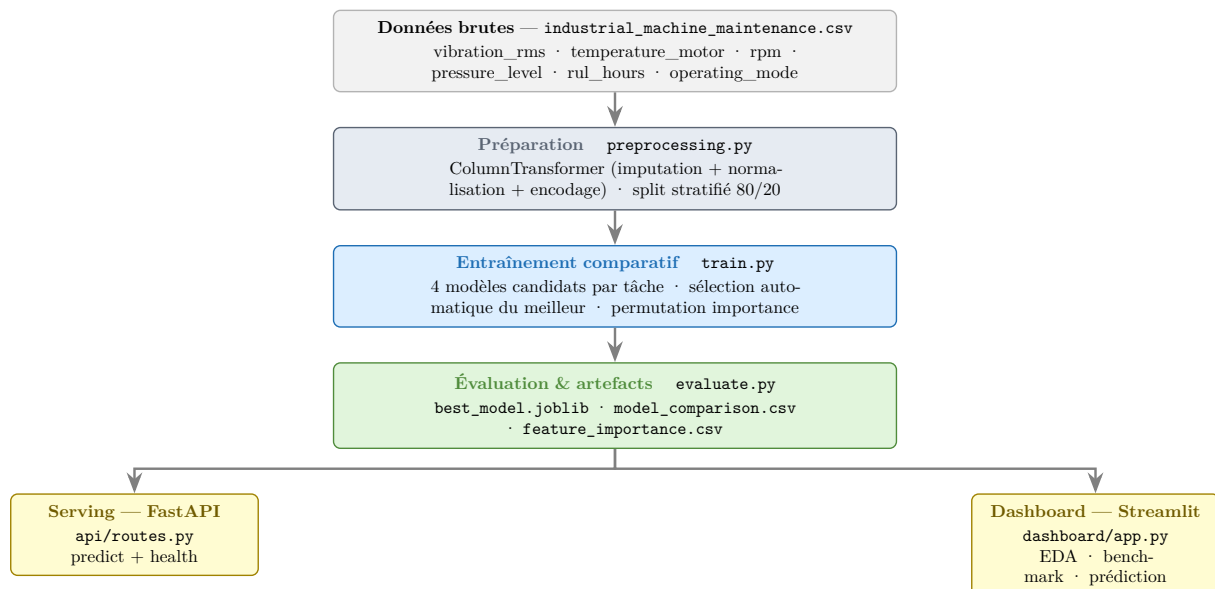
Cette prédiction déclenche une intervention préventive ciblée avant la défaillance.

- **Classification multiclasse (5 classes)** — identifier le type de panne attendu. Cette prédiction oriente le technicien vers la pièce ou le sous-système en cause, réduisant le temps de diagnostic.

L'enjeu industriel est direct et chiffrable. Une fausse alerte mobilise une équipe de maintenance pour rien ; un faux négatif laisse passer une panne réelle qui peut coûter plusieurs heures d'arrêt de production. Cet arbitrage asymétrique justifie le choix de la métrique primaire : le **F1-score** (moyenne harmonique de la précision et du rappel), qui pénalise simultanément les fausses alertes et les pannes manquées.

3 Architecture et cycle de vie du modèle

L'architecture du projet suit le cycle de vie standard d'un projet ML, de la donnée brute jusqu'à la mise en service. Chaque étape est isolée dans un module dédié, ce qui garantit la testabilité et la reproductibilité de l'ensemble.



Ce découpage modulaire est un choix d'ingénierie : la préparation, l'entraînement, l'évaluation et le serving sont indépendants. On peut réentraîner les modèles sans toucher à l'API, ou faire évoluer le tableau de bord sans réentraîner. Il est impératif de séparer les responsabilités pour que le projet ML puisse être maintenu.

4 Analyse exploratoire et compréhension métier

La maintenance industrielle suit historiquement deux modèles. La maintenance *corrective* répare après la panne : elle maximise l'usure utile des pièces mais expose à des arrêts non planifiés et coûteux. La maintenance *préventive planifiée* révisé à intervalle fixe : elle est sûre mais remplace des pièces encore fonctionnelles, gaspillant de la valeur.

La maintenance *prédictive* vise un troisième modèle, supérieur aux deux précédents : intervenir au bon moment, ni trop tôt ni trop tard, en se fondant sur l'état réel de la machine mesuré par les capteurs. C'est précisément l'objet des deux tâches de prédiction du projet.

Stratégie	Pré-requis	Coût d'une panne	Coût de maintenance
Corrective	Aucun	Élevé (arrêt non prévu)	Faible (minimal)
Préventive planifiée	Calendrier	Faible	Élevé (pièces saines remplacées)
Prédictive	Capteurs + ML	Quasi-nul	Optimisé (juste à temps)

L'indicateur `rul_hours` (*Remaining Useful Life*, durée de vie résiduelle) est au cœur du problème : il estime le temps restant avant la prochaine défaillance. Comme le confirmera l'analyse d'importance des variables, cet indicateur domine la prédiction — un résultat cohérent avec l'intuition métier.

4.1 Exploration des données

Le jeu de données regroupe 24 042 mesures de capteurs. L'analyse exploratoire (EDA) précède la modélisation : elle révèle le fort déséquilibre des cibles et la séparation des distributions de capteurs selon le statut de panne, deux observations qui orientent les choix méthodologiques.

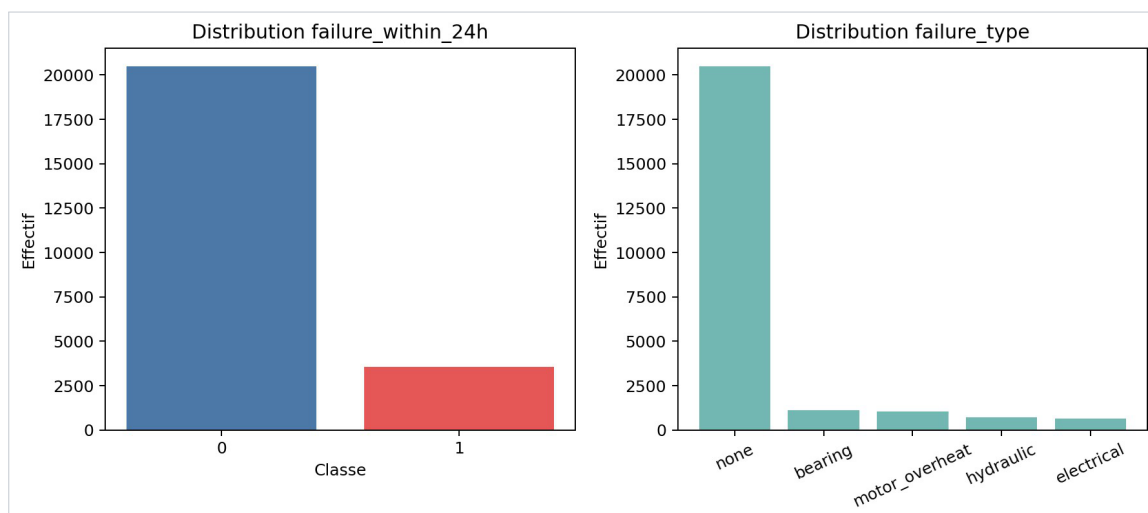


Figure 1 – Distribution des deux cibles. La classe positive `failure_within_24h=1` ne représente que 14,8 % des observations, et la cible `failure_type` est dominée par la classe `none` (85 %). Ce déséquilibre impose le *F1-score* et la *PR-AUC* comme métriques, ainsi qu'un *split stratifié*.

Les boîtes à moustaches confirment que les variables dominantes séparent nettement les machines saines des machines à risque : sur les machines en panne, la température moyenne passe de 49 à 63 °C et la durée de vie résiduelle chute de 30,6 à 11,8 h.

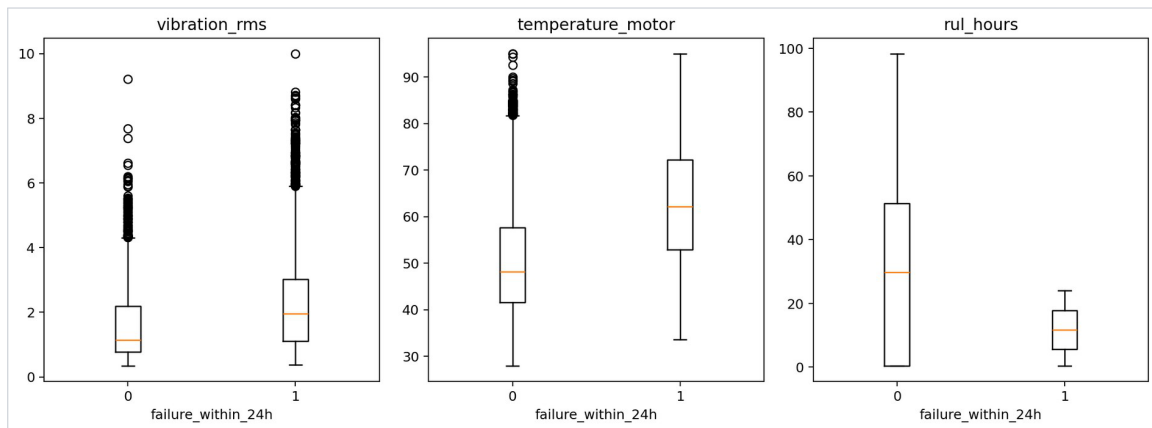


Figure 2 – Distribution des variables clés selon le statut de panne. La séparation nette des médianes (température, vibration, RUL) justifie leur pouvoir prédictif observé plus loin.

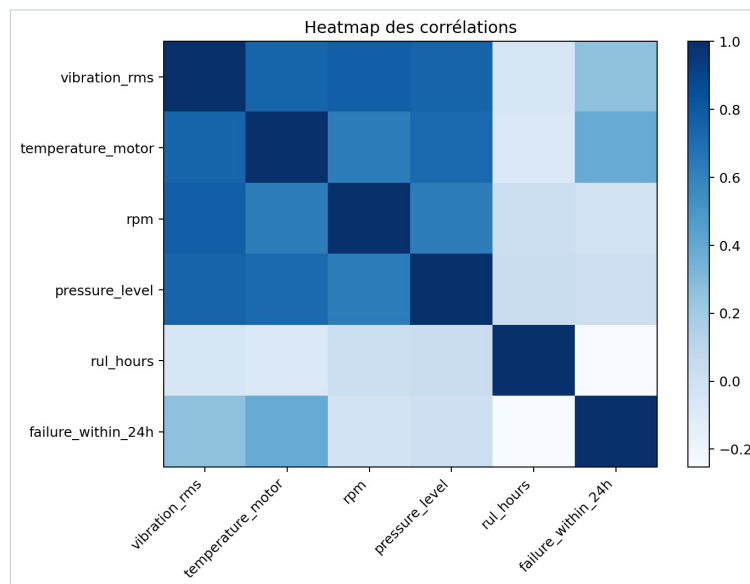


Figure 3 – Carte de chaleur des corrélations. La température moteur (+0,39), la vibration (+0,26) et la durée de vie résiduelle (-0,25) sont les plus corrélées à la cible binaire.

5 Préparation des données et pipeline de transformation

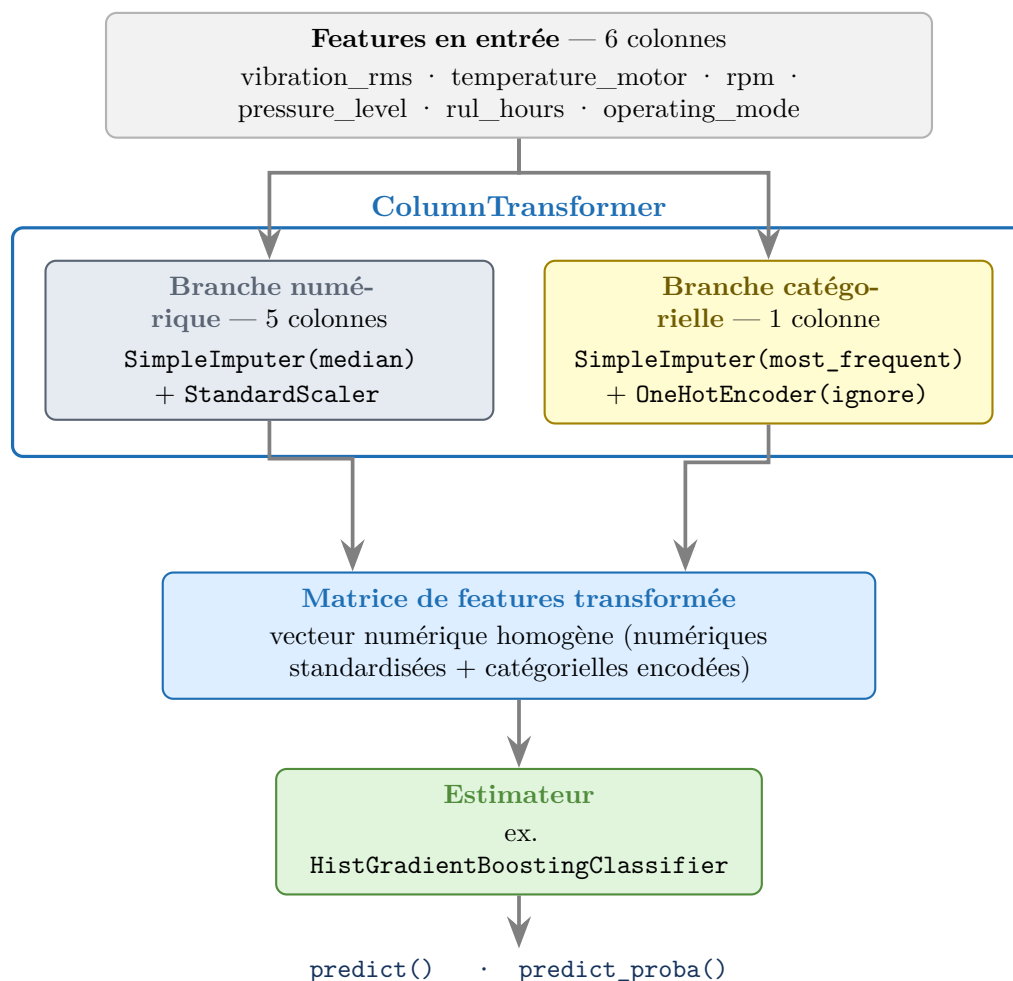
5.1 Variables explicatives et cibles

Le jeu de données contient six variables explicatives (cinq numériques, une catégorielle) et deux cibles alternatives, une par tâche de prédiction.

Variable	Type	Description
rul_hours	Numérique	Heures de vie résiduelle estimée
temperature_motor	Numérique	Température moteur (°C)
rpm	Numérique	Vitesse de rotation (tours/min)
vibration_rms	Numérique	Amplitude vibratoire RMS
pressure_level	Numérique	Niveau de pression
operating_mode	Catégoriel	Mode de fonctionnement
failure_within_24h	Cible binaire	Une panne surviendra-t-elle sous 24 h ?
failure_type	Cible multiclasse	Type de panne (5 classes)

5.2 Le pipeline de transformation : ColumnTransformer

Les variables numériques et la variable catégorielle exigent des traitements distincts. Le **ColumnTransformer** de scikit-learn applique deux sous-pipelines en parallèle, puis concatène leurs sorties en une matrice de features homogène prête pour l'estimateur. Le schéma ci-dessous illustre ce flux.



```
def build_preprocessor() -> ColumnTransformer:
    numeric_pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ])
    categorical_pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore")),
    ])
    return ColumnTransformer(transformers=[
        ("numeric", numeric_pipeline, NUMERIC_COLUMNS),
        ("categorical", categorical_pipeline, CATEGORICAL_COLUMNS),
    ])

def make_train_test_split(features, target, test_size=0.2):
    validate_stratified_split_capacity(target, test_size)
    return train_test_split(
        features, target,
        test_size=test_size, random_state=RANDOM_STATE,
        stratify=target, # garantit la representation de toutes les classes
    )
```

Extrait 1 – Pipeline de préparation – ColumnTransformer et split stratifié (preprocessing.py)

Chaque choix de transformation répond à une contrainte du domaine. L'imputation par la **médiane** pour les variables numériques est robuste aux valeurs aberrantes fréquentes sur des capteurs. L'option `handle_unknown="ignore"` de l'encodeur prévient toute erreur à l'inférence si un mode opératoire inconnu apparaît en production : la ligne est encodée par un vecteur nul plutôt que de provoquer une exception. Enfin, la **stratification** du split garantit que chaque classe — y compris les pannes rares — est représentée dans le jeu de test avec la même proportion qu'à l'entraînement, condition d'une évaluation statistiquement valide.

La taille du jeu de test est par ailleurs calculée dynamiquement par `test_size = max(0.2, n_classes / n_rows)`. Ce garde-fou empêche qu'une division 80/20 sur un jeu restreint produise un jeu de test trop petit pour contenir au moins un exemple de chaque classe.

6 Entraînement comparatif et sélection du modèle

6.1 Architecture du benchmark

Le principe directeur est l'**étanchéité** entre l'entraînement et l'évaluation. Le préprocesseur et le modèle sont encapsulés dans un unique `Pipeline` scikit-learn : le `StandardScaler` et les imputeurs sont ajustés uniquement sur les données d'entraînement, jamais sur le jeu de test. Cette discipline élimine toute fuite de données (*data leakage*) qui gonflerait artificiellement les scores.

```

1 for model_name, estimator in candidate_models.items():
2     pipeline = Pipeline(steps=[
3         ("preprocessor", build_preprocessor()),
4         ("model", estimator),
5     ])
6     pipeline.fit(x_train, y_train)
7     predicted_labels = pipeline.predict(x_test)
8     predicted_probabilities = pipeline.predict_proba(x_test)
9
10    evaluation_rows.append(evaluate_classifier(
11        task_name=task_name, model_name=model_name,
12        true_values=y_test,
13        predicted_labels=predicted_labels,
14        predicted_probabilities=predicted_probabilities,
15    ))
16
17 comparison = compare_results(task_name=task_name, results=evaluation_rows)
18 best_row = select_best_model(comparison) # tri sur la metrique primaire
19 best_pipeline = trained_pipelines[best_row["model_name"]]
20 save_model(best_pipeline, model_path) # serialisation joblib

```

Extrait 2 – Boucle de benchmark et sélection automatique (train.py)

La sélection du meilleur modèle est entièrement automatisée : les résultats sont triés sur la métrique primaire (F1 en binaire, weighted_F1 en multiclasse) et le vainqueur est sérialisé en joblib pour la mise en service. Aucune intervention manuelle n'intervient dans le choix final, ce qui rend la démarche reproductible.

6.2 Modèles candidats

Quatre familles de modèles sont mises en concurrence pour chaque tâche, couvrant un spectre volontairement large : un modèle linéaire (Logistic Regression), deux modèles d'ensemble (Random Forest, gradient boosting) et un réseau de neurones (Multilayer Perceptron).

```

1 # Tache binaire (failure_within_24h)
2 {
3     "logistic_regression": LogisticRegression(max_iter=1000, random_state=42),
4     "random_forest": RandomForestClassifier(
5         n_estimators=300, random_state=42,
6         class_weight="balanced"), # compense le
7         desequilibre
8     "gradient_boosting": HistGradientBoostingClassifier(random_state=42),
9     "mlp_classifier": MLPClassifier(
10         hidden_layer_sizes=(64, 32),
11         max_iter=300, random_state=42),
12 }

```

Extrait 3 – Modèles candidats par tâche (models.py)

L'option `class_weight="balanced"` de la Random Forest compense le déséquilibre de classes inhérent aux données de maintenance — les pannes sont rares par définition. Le `HistGradientBoostingClassifier` gère nativement les valeurs manquantes, sans imputation préalable, ce qui en fait un candidat robuste même lorsqu'un capteur perd des mesures.

6.3 Le modèle vainqueur : HistGradientBoostingClassifier

Le `HistGradientBoostingClassifier` est l'implémentation scikit-learn du gradient boosting (approche popularisée par LightGBM). Il se distingue par deux optimisations déterminantes :

1. **Discrétisation par histogrammes.** Plutôt que de tester tous les seuils de division possibles, l'algorithme regroupe chaque variable en 256 intervalles (bins) et ne teste que ces 256 valeurs. La complexité d'entraînement chute de $O(n \times d)$ à $O(256 \times d)$, où n est le nombre d'exemples et d le nombre de variables.
2. **Gestion native des valeurs manquantes.** L'algorithme apprend, pour chaque division, si une valeur manquante doit suivre la branche gauche ou droite. Cette propriété est directement utile face à des capteurs pouvant perdre des mesures (panne capteur, coupure réseau).

Caractéristique	HistGradientBoosting	RandomForest
Valeurs manquantes	Nativement gérées	Imputation requise
Complexité d'entraînement	$O(256 \times d \times \text{arbres})$	$O(n \log n \times \text{arbres})$
Risque de surapprentissage	Modéré (régularisation ℓ_2)	Faible (bagging)
Interprétabilité	Faible (boîte noire)	Moyenne (importance native)
Gestion du déséquilibre	Bonne (<code>class_weight</code>)	Très bonne (<code>balanced</code>)

Comme le montreront les résultats, aucun algorithme n'est universellement supérieur : le Gradient Boosting l'emporte sur la tâche binaire, tandis que la Random Forest domine la tâche multiclasse déséquilibrée. Le benchmark systématique est donc indispensable pour choisir le bon modèle par cas d'usage.

7 Résultats et évaluation

7.1 Tâche 1 — Classification binaire `failure_within_24h`

Modèle	F1	Précision	Rappel	ROC-AUC	PR-AUC
gradient boosting	0,9775	0,9789	0,9761	0,9991	0,9956
random forest	0,9752	0,9857	0,9649	0,9995	0,9976
mlp classifieur	0,9337	0,9697	0,9003	0,9974	0,9877
logistic regression	0,7224	0,8456	0,6306	0,9485	0,8273

Le **Gradient Boosting** remporte le benchmark sur la métrique primaire F1 (0,9775), avec le meilleur équilibre précision/rappel. La Random Forest obtient une précision et une ROC-AUC légèrement supérieures, mais son rappel plus faible (0,9649) — donc davantage de pannes manquées — pénalise son F1. La Logistic Regression confirme la nature non linéaire du problème : son rappel de 0,6306 signifie qu'elle manque près de 37 % des pannes réelles, ce qui la rend inutilisable pour ce cas d'usage critique.

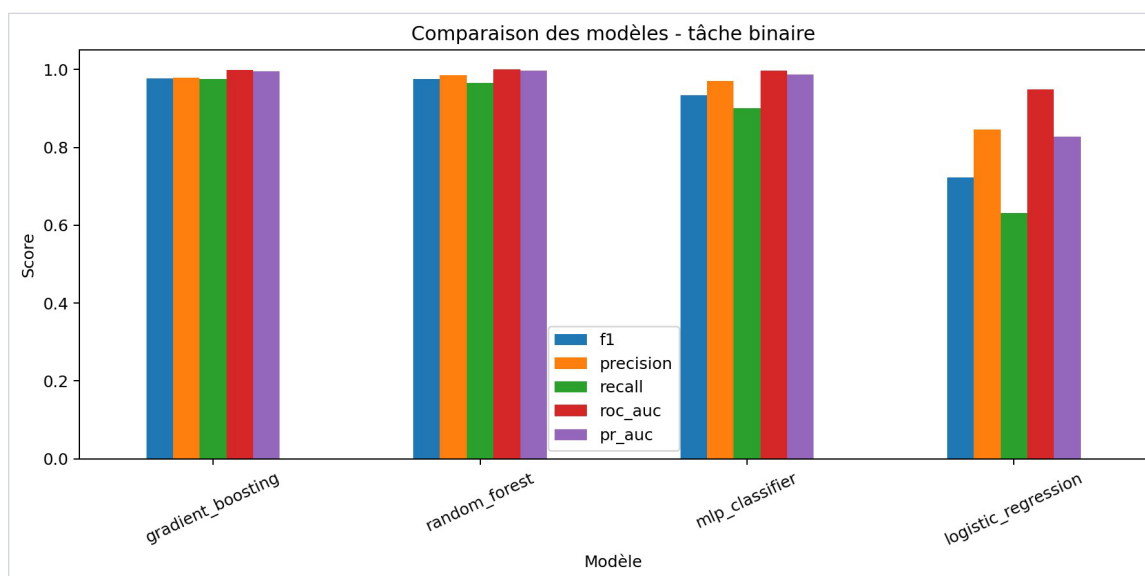


Figure 4 – Comparaison visuelle des quatre modèles sur la tâche binaire (F1, précision, rappel, ROC-AUC, PR-AUC). Les deux modèles d'ensemble dominent ; la régression logistique décroche nettement sur le rappel et la PR-AUC.

7.2 Tâche 2 — Classification multiclasse `failure_type` (5 classes)

Modèle	weighted_F1	macro_F1	Classes
random_forest	0,9782	0,8955	5
hist_gradient_boosting	0,9753	0,8829	5

Sur la tâche multiclasse, la Random Forest prend la tête (weighted_F1 = 0,9782). Le macro_F1 de 0,8955 — moyenne non pondérée par la taille des classes — indique que les classes minoritaires restent globalement bien gérées. L'écart entre weighted_F1 (0,9782) et macro_F1 (0,8955) révèle néanmoins que certaines classes rares sont prédites avec moins de fiabilité : c'est l'axe d'amélioration principal, détaillé en section 9.

7.3 Interprétabilité — importance des variables

L'interprétabilité est essentielle pour qu'un modèle soit accepté en production par les équipes métier. Nous calculons l'**importance par permutation** sur le jeu de test : pour chaque variable, on mesure la chute du F1 lorsque ses valeurs sont mélangées aléatoirement. Plus la chute est forte, plus la variable est prédictive.

Variable	Importance (permutation)
rul_hours	0,4954
temperature_motor	0,2899
rpm	0,2772
vibration_rms	0,1165
pressure_level	0,0471
operating_mode	0,0174

La variable `rul_hours` domine très nettement (0,495) : elle est le signal le plus prédictif d'une panne imminente, ce qui valide l'intuition métier. La température moteur (0,290) et la vitesse de rotation (0,277) sont les deux facteurs physiques complémentaires. À l'inverse, le mode opératoire est quasi nul (0,017) : il n'apporte pas d'information prédictive significative sur ce jeu de données. Ce classement est directement exploitable : il indique aux équipes quelles grandeurs instrumenter en priorité.

```

importance_result = permutation_importance(
    estimator=best_pipeline,
    X=x_test, y=y_test,
    n_repeats=10,
    random_state=RANDOM_STATE,
    scoring="f1_weighted",
)
importance_frame = pd.DataFrame({
    "feature": x_test.columns,
    "importance_mean": importance_result.importances_mean,
    "importance_std": importance_result.importances_std,
}).sort_values(by="importance_mean", ascending=False)

```

Extrait 4 – Calcul de L'importance par permutation (train.py)

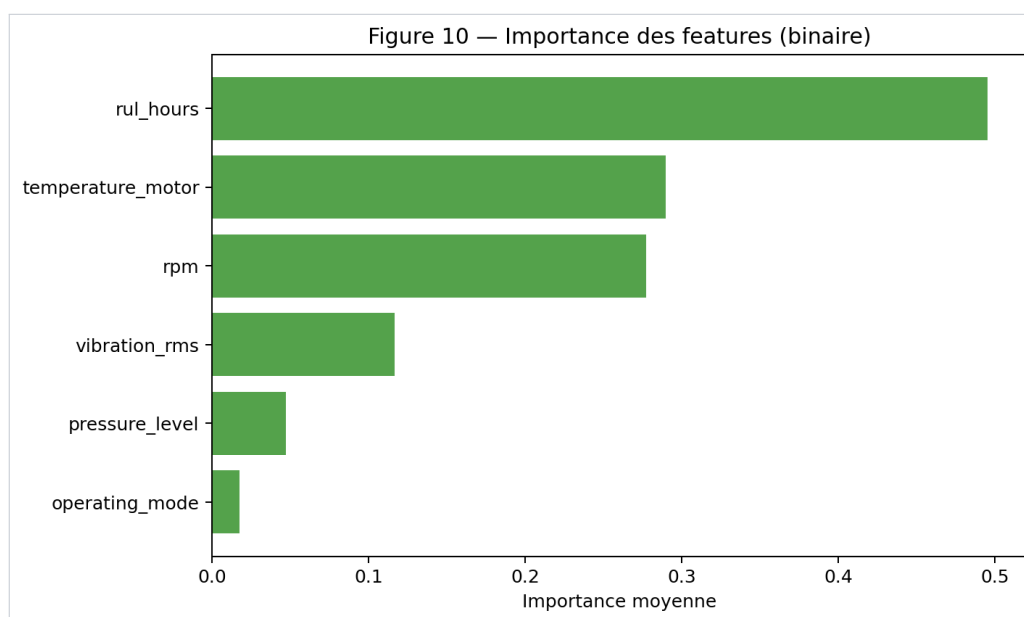


Figure 5 – Importance des variables par permutation (tâche binaire). La durée de vie résiduelle `rul_hours` explique à elle seule près de la moitié de la performance du modèle.

7.4 Analyse des erreurs — matrice de confusion

Au-delà des scores agrégés, l'analyse de la matrice de confusion éclaire le comportement réel du modèle face aux deux types d'erreurs.

	Prédit : panne	Prédit : pas de panne
Réel : panne	VP (vrais positifs)	FN (faux négatifs — risque métier)
Réel : pas de panne	FP (faux positifs — coût mobilisation)	VN (vrais négatifs)

En maintenance industrielle, les deux erreurs n'ont pas le même coût : un faux négatif (panne manquée) peut entraîner un arrêt de production, tandis qu'un faux positif (fausse alerte) ne mobilise qu'une équipe pour une vérification inutile. Le modèle retenu affiche un rappel de 0,9761 (97,6 % des pannes réelles détectées) pour une précision de 0,9789 : l'équilibre penche légèrement en faveur du rappel, ce qui est exactement le comportement souhaité pour ce cas d'usage.

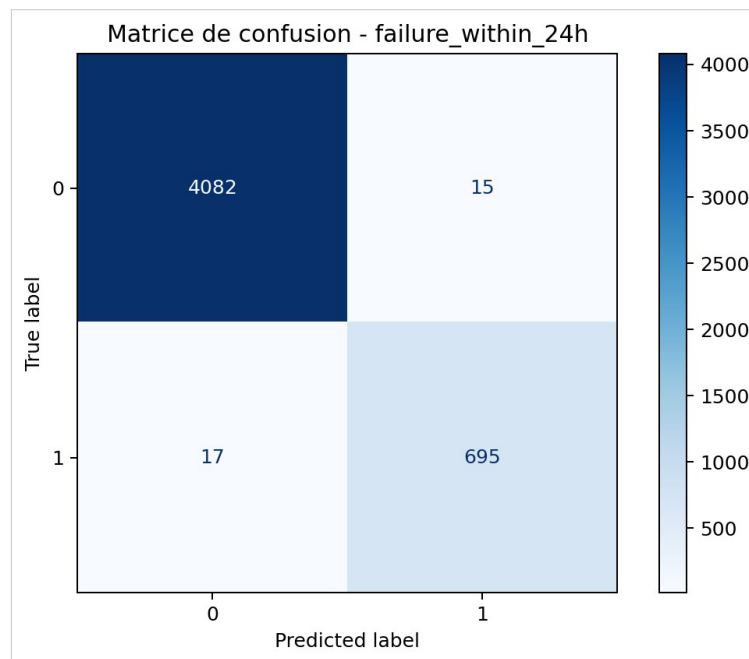


Figure 6 – Matrice de confusion de la tâche binaire (modèle *gradient_boosting*). Sur le jeu de test, seuls 17 faux négatifs et 15 faux positifs subsistent face à 4 082 vrais négatifs et 695 vrais positifs.

Sur la tâche multiclasse, la matrice de confusion révèle que la classe majoritaire **none** est presque parfaitement identifiée, tandis que les confusions résiduelles se concentrent entre types de pannes rares (**electrical**, **hydraulic**) — ce qui explique l'écart entre **weighted_F1** et **macro_F1**.

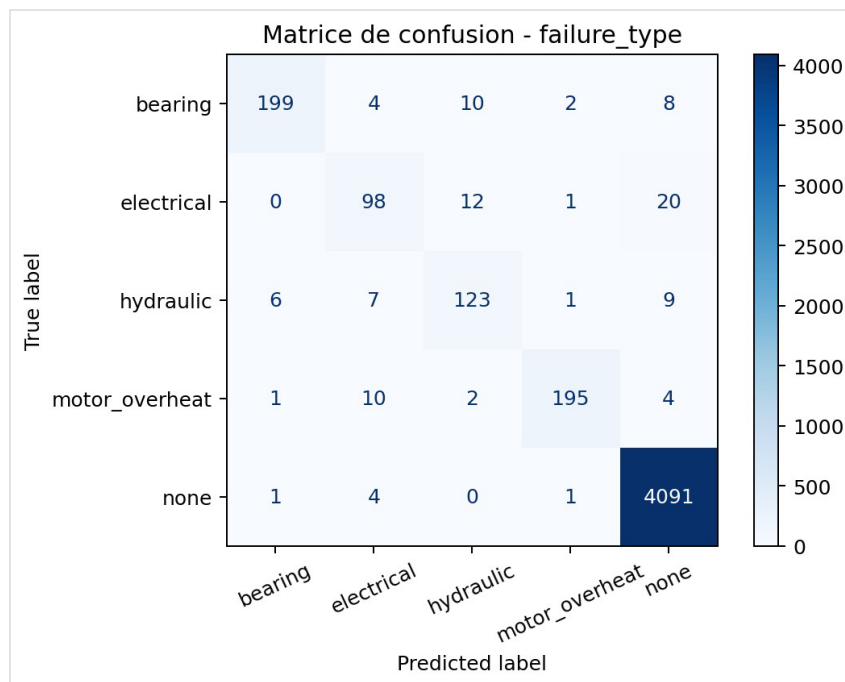


Figure 7 – Matrice de confusion de la tâche multiclassée (modèle *random_forest*, 5 classes). La diagonale dominante confirme la qualité globale ; les erreurs se concentrent sur les classes minoritaires.

8 Mise en service et communication des résultats

8.1 API de prédiction — FastAPI

Les modèles sérialisés sont exposés via une API REST construite avec FastAPI [3]. Chaque tâche dispose de son endpoint de prédiction, et un endpoint de supervision expose l'état de chargement des artefacts.

```
@router.post("/predict/failure-within-24h",
              response_model=FailureWithin24hPredictionResponse)
def predict_binary(payload: FailureWithin24hPredictionRequest):
    return predict_failure_within_24h(payload.model_dump())

@router.post("/predict/failure-type",
              response_model=FailureTypePredictionResponse)
def predict_multiclass(payload: FailureTypePredictionRequest):
    return predict_failure_type(payload.model_dump())

@router.get("/health", response_model=HealthResponse)
def health() -> HealthResponse:
    available_tasks = []
    for task_name in config.TASK_CONFIGS:
        try:
            load_task_bundle(task_name)
        except TaskArtifactError:
            continue
        available_tasks.append(task_name)
    status = "ok" if len(available_tasks) == len(config.TASK_CONFIGS) else "degraded"
    return HealthResponse(status=status, available_tasks=available_tasks)
```

Extrait 5 – Endpoints de prédiction et supervision (*api/routes.py*)

L'endpoint `/health` renvoie "ok" uniquement si les deux artefacts (binaire et multiclassé) sont

chargés ; sinon "degraded". Ce mécanisme permet un monitoring en production. La validation des entrées est assurée par Pydantic v2 via les schémas `FailureWithin24hPredictionRequest` et `FailureTypePredictionRequest`, cohérents avec les six variables déclarées dans `config.py`.

8.2 Tableau de bord interactif — Streamlit

Le tableau de bord Streamlit [4] (`dashboard/app.py`) répond directement à la sous-compétence de « communication infographique visuelle ». Il s'organise en trois pages complémentaires :

- **Page exploration (EDA)** : distributions des variables capteurs (histogrammes, boîtes à moustaches), corrélations entre variables (carte de chaleur), répartition des classes cibles. Elle permet à un expert métier de comprendre les données sans écrire de code.
- **Page benchmark** : tableau comparatif des modèles (F1, précision, rappel, ROC-AUC), graphique des importances de variables, sélection interactive du modèle visualisé.
- **Page prédiction** : formulaire de saisie des six variables capteurs, appel en temps réel à l'API FastAPI, affichage de la prédiction et de la probabilité associée.

La page de prédiction ne ré-embarque pas le modèle : elle consomme l'API. Ainsi, tout déploiement d'un nouveau modèle est immédiatement reflété dans le tableau de bord sans le redémarrer — une conséquence directe de l'architecture découplée.

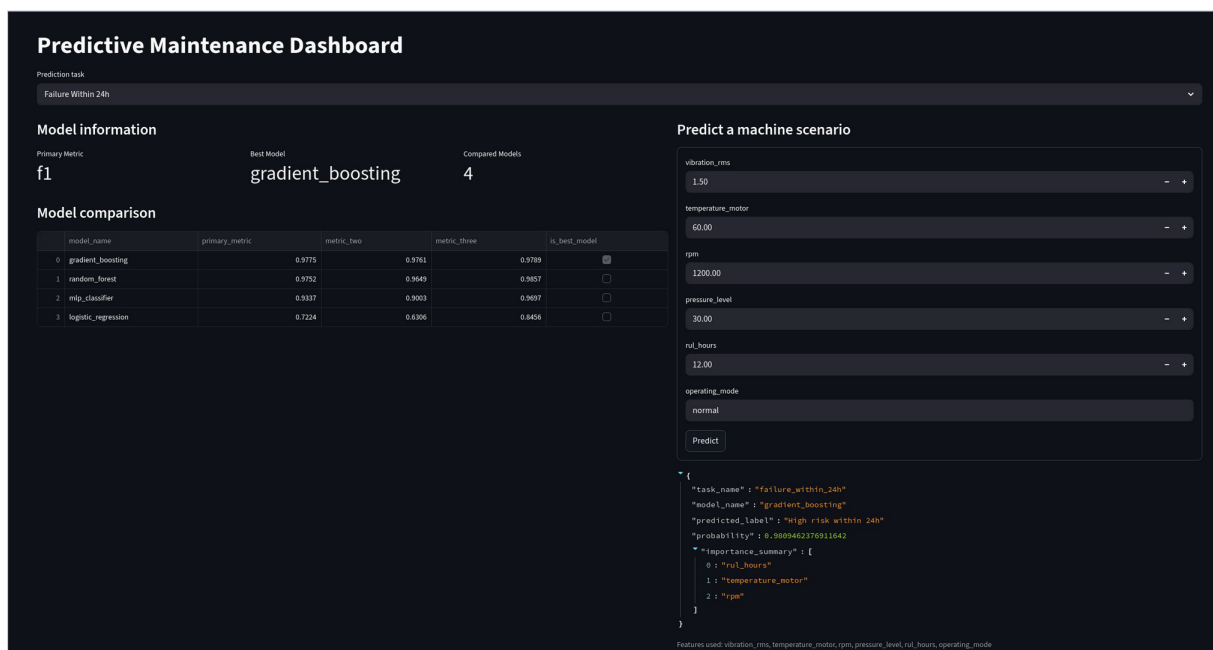


Figure 8 – Tableau de bord Streamlit. À gauche : informations du modèle servi et comparaison des modèles. À droite : formulaire de saisie des six variables capteurs et réponse JSON de l'API (probabilité, label, top-3 des variables influentes).

8.3 Qualité logicielle et reproductibilité

L'ensemble du pipeline est couvert par des tests automatisés `pytest`, module par module, garantissant que chaque étape est validée indépendamment. Cette couverture est la condition de la reproductibilité des résultats annoncés.

Module testé	Périmètre de validation
test_data.py	Chargement du dataset, séparation features/cible
test_preprocessing.py	Pipeline ColumnTransformer, split stratifié
test_prediction_pipeline.py	Pipeline complet entraînement → prédiction
test_evaluate.py	Métriques binaires et multiclasses
test_explainability.py	Permutation importance
test_api.py	Endpoints FastAPI (health, predict)
test_dashboard_helpers.py	Fonctions de visualisation Streamlit

La configuration des tâches est par ailleurs centralisée dans un unique dictionnaire `TASK_CONFIGS` (`config.py`), source de vérité partagée par les modules d'entraînement, d'évaluation et de serving. Ajouter une nouvelle tâche se résume à y ajouter une entrée : aucun chemin n'est codé en dur dans les modules métier.

9 Robustesse, limites et axes d'amélioration

9.1 Validation : split stratifié et perspective k-fold

La version actuelle repose sur un split stratifié 80/20, choisi pour son rapport coût/robustesse : huit entraînements complets (quatre modèles × deux tâches) restent rapides. Une validation croisée *k-fold* ($k=5$) fournirait une estimation plus robuste de la variance, au prix d'un temps de calcul multiplié par cinq.

Méthode	Avantage	Inconvénient
Split 80/20 stratifié	Rapide, simple à implémenter	Variance selon la graine aléatoire
k-fold (k=5)	Estimation robuste (k résultats)	5× le temps de calcul
Stratified k-fold	Robustesse + représentation des classes	Même coût que k-fold

9.2 Limites de l'interprétabilité par permutation

L'importance par permutation reste une méthode globale qui présente deux limites. D'une part, en présence de variables corrélées (par exemple `rul_hours` et `temperature_motor`), permuter l'une laisse l'information accessible via l'autre, sous-estimant l'importance des deux. D'autre part, son coût croît avec le produit du nombre de variables par `n_repeats`. Une approche complémentaire serait SHAP (*SHapley Additive exPlanations*), qui fournit une interprétabilité **locale**, instance par instance, en plus de la vue globale.

9.3 Axes d'amélioration

L'écart entre `macro_F1` (0,8955) et `weighted_F1` (0,9782) sur la tâche multiclasse indique que les classes de pannes rares sont sous-représentées. Deux leviers sont identifiés : un suréchantillonnage **SMOTE** (génération d'exemples synthétiques pour les classes minoritaires par interpolation) et un ajustement des poids de classe à l'entraînement multiclasse. Enfin, l'ajout d'un réentraînement périodique piloté par une détection de dérive (*drift detection*) permettrait au modèle de s'adapter à l'évolution du comportement des machines en production.

10 Synthèse — compétences démontrées

Le **projet data science** couvre l'intégralité du périmètre du Bloc 4, de la donnée brute à la mise en service, en démontrant chacune des cinq sous-compétences du référentiel.

10.1 Preuves d'acquisition des compétences

Le tableau suivant récapitule, pour chacune des cinq sous-compétences du Bloc 4, la réalisation concrète qui la démontre dans le projet et le résultat mesurable obtenu.

Compétence visée	Réalisation et preuve dans le projet
Extraire des données de systèmes variés	Chargement et exploration d'un jeu de données de capteurs industriels (<code>data.py</code> , 6 variables, 2 cibles) avec séparation features/cible contrôlée.
Préparer et nettoyer les données	Pipeline <code>ColumnTransformer</code> : imputation médiane + standardisation (numériques), imputation modale + encodage one-hot (catégoriel), split stratifié 80/20.
Communication infographique (tableaux de bord interactifs)	Tableau de bord Streamlit à trois pages : exploration (EDA), comparaison des modèles, prédiction en temps réel via l'API.
Développer un modèle prédictif supervisé	Deux tâches (classification binaire et multiclasse à 5 classes), quatre modèles candidats par tâche, sélection automatisée du meilleur.
Évaluer et comparer les performances	Benchmark sur F1, précision, rappel, ROC-AUC, PR-AUC ; meilleur F1 = 0,9775 (binaire) et <code>weighted_F1</code> = 0,9782 (multiclasse).

Références

- [1] France Compétences. *Fiche RNCP36739 — Expert en ingénierie de données*. Disponible sur : <https://www.francecompetences.fr/recherche/rncp/36739/> [consulté le 15 juin 2026].
- [2] Pedregosa F., Varoquaux G., Gramfort A. et al. *Scikit-learn : Machine Learning in Python*. *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [3] Ramírez S. *FastAPI — modern, fast web framework for building APIs with Python*. Documentation officielle. Disponible sur : <https://fastapi.tiangolo.com/> [consulté le 15 juin 2026].
- [4] Streamlit Inc. *Streamlit — A faster way to build and share data apps*. Documentation officielle. Disponible sur : <https://docs.streamlit.io/> [consulté le 15 juin 2026].